

گزارش پروژه شماره 2: جستجوی محلی (آخرین سیگنال)

درس هوش مصنوعی، نیم سال دوم 1404-1405، دانشگاه اصفهان

استاد: دکتر فریا نصیری مفخم

اعضای گروه: مریم موسوی و ستایش اسدیان فیلی

<https://github.com/SmithyOrFalse/Project2>

لینک رپازیتوری گیت (پابلیک):

1. شرح مسئله

است: چیدمان مجموعه‌ای از سنسورها در Grid World هدف پروژه، حل یک مسئله بهینه‌سازی در محیط یک نقشه‌ی شطرنجی به گونه‌ای که بیشترین تعداد اهداف زیر پوشش قرار گیرد، سنسورها روی موانع یا خارج محیط قرار نگیرند، و تعداد سنسورهای مورد استفاده تا حد ممکن کم باشد. شعاع پوشش هر سنسور با فاصله‌ی منتهن سنجیده می‌شود (مطابق با تابع رندر پروژه).

این یک مسئله ترکیبی با فضای حالت گسسته و بزرگ است؛ بنابراین الگوریتم‌های جستجوی محلی انتخاب (Hill Climbing و Simulated Annealing) طبیعی برای حل آن هستند. در این پروژه دو الگوریتم اجباری پیاده‌سازی و با یکدیگر (Genetic Algorithm, Beam Search, Tabu Search) و سه الگوریتم امتیازی مقایسه شده‌اند.

2. ساختار کد

کد پروژه روی اسکلت داده‌شده توسط استاد بنا شده است. فقط در پوشه‌ی `/search` و فایل `main.py` تغییر/افزودن انجام شده است:

```
project/
├─ main.py # نمایش نتایج و اجرای الگوریتم‌ها و نمایش
├─ utils.py # توابع نمایش/رسم (بدون تغییر)
├─ env/ # GridWorld محیط و رندر (بدون تغییر)
└─ search/
```

```
└─ local_search_base.py # کلاس پایه: evaluate / get_neighbor / initialize_state
└─ hill_climbing.py # الگوریتم Hill Climbing
└─ simulated_annealing.py # الگوریتم Simulated Annealing
└─ genetic_algorithm.py # امتیازی
└─ beam_search.py # امتیازی
└─ tabu_search.py # امتیازی
```

3. طراحی تابع هزینه (Cost Function)

تابع هزینه‌ی پیاده‌سازی‌شده، یک ترکیب وزن‌دار از سه عبارت است (در فایل `local_search_base.py`):

```
cost(state) = W_UNCOVERED * (#uncovered targets)
              + W_SENSOR * (#sensors used)
              + W_OVERLAP * (extra coverers beyond the first per target)

W_UNCOVERED = 100
W_SENSOR = 5
W_OVERLAP = 1
```

منطق انتخاب وزن‌ها:

- دارای وزن غالب است تا اولویت اول الگوریتم همواره «پوشش دادن اهداف» `uncovered` عبارت باشد.
- با وزن کوچک‌تر، حالت‌هایی را که با تعداد سنسور کمتر همان پوشش را می‌دهند `sensor` عبارت خواسته‌شده در صورت پروژه `trade-off` ترجیح می‌دهد دقیقاً همان دو سنسوری را که یک هدف مشترک را پوشش می‌دهند جریمه می‌کند، که عملاً `overlap` عبارت سنسورها را به سمت پراکندگی روی نقشه می‌راند و گیر افتادن در چیدمان‌های متراکم را کاهش می‌دهد.

چون «هزینه‌ی کمتر = بهتر»، الگوریتم‌ها همگی به‌صورت `minimization` پیاده شده‌اند.

4. طراحی تابع همسایگی (Neighbor Function)

در تولید همسایه پشتیبانی `remove` و `add` و `move` صورت پروژه به‌صراحت می‌خواهد که هر سه عملیات

شوند. این عملیات‌ها در کلاس پایه پیاده‌سازی شده‌اند:

• **MOVE:** یک سنسور به صورت تصادفی انتخاب می‌شود؛ ابتدا تلاش می‌شود به یکی از چهار خانه‌ی مجاور (همسایگی فون نیومن) منتقل شود (برای fine-tuning). اگر همه‌ی خانه‌های مجاور مانع یا اشغال بودند، سنسور به یک خانه‌ی معتبر تصادفی در کل نقشه پرش می‌کند (برای فرار از local optimum).

• **ADD:** یک خانه‌ی خالی و معتبر تصادفی انتخاب و سنسوری در آن قرار می‌گیرد مشروط بر اینکه سقف max_sensors رعایت شود.

• **REMOVE:** یک سنسور تصادفی حذف می‌شود مشروط بر اینکه دست‌کم یک سنسور باقی بماند.

هرکدام با وزن 1. دلیل این انتخاب آن است که remove و add، با وزن 3 move: احتمال انتخاب هر عملیات وقتی تعداد سنسورها تقریباً درست باشد، عمده‌ی پیشرفت‌های ممکن از طریق جابه‌جا کردن سنسورهای به مرز state به علاوه، در حالتی که اندازه‌ی state موجود به دست می‌آیند نه از طریق تغییر اندازه‌ی ظرفیت رسیده باشد، عملیات نامعتبر به طور خودکار از فهرست انتخاب حذف می‌شود.

این طراحی فضای جستجو را پویا می‌کند یعنی تعداد سنسورها در طول اجرا تغییر می‌کند و این دقیقاً همان رفتاری است که پروژه برای مدیریت بهینه‌ی trade-off کیفیت پوشش و مصرف منابع از ما خواسته است.

5. الگوریتم Hill Climbing

پیاده‌سازی این الگوریتم در فایل search/hill_climbing.py از نوع **steepest-ascent تصادفی** با دو افزونه است:

1. در هر مرحله، تعداد ثابتی همسایه (پیش‌فرض 25) نمونه‌برداری شده و کم‌هزینه‌ترین آن‌ها انتخاب می‌شود.

2. **حرکت‌های sideways:** اگر بهترین همسایه هم‌هزینه با حالت فعلی باشد، تا سقف max_sideways=10 گام به آن منتقل می‌شویم تا plateau‌های صاف (که در این مسئله بسیار رایج‌اند، چون هزینه‌ها گسسته هستند) رد شوند.

3. **راه‌اندازی مجدد تصادفی:** اگر گیر کنیم و بودجه‌ی sideways تمام شود، تا max_restarts=3 بار از یک حالت تصادفی جدید شروع می‌کنیم، در حالی که بهترین حالت دیده‌شده تاکنون را نگه می‌داریم.

را تا حد قابل توجهی local optimum پایه یعنی گیر کردن در Hill Climbing این دو افزونه ضعف اصلی کاهش می‌دهند، بدون اینکه ماهیت «بهترین همسایه را بگیر» الگوریتم تغییر کند.

6. الگوریتم Simulated Annealing

پیاده‌سازی استاندارد با کوولینگ نمایی (در فایل `search/simulated_annealing.py`):

```
T_{t+1} = T_t * alpha, alpha = 0.995
P(accept worse) = exp(-delta / T)
T_start = 10.0, T_min = 1e-3, max_iter = 2000
```

در ابتدای اجرا (دمای بالا) تقریباً هر همسایه‌ای حتی اگر بدتر باشد قبول می‌شود، که اکتشاف وسیع فضای حالت را ممکن می‌کند. به مرور دما پایین می‌آید و فقط بهبودها قبول می‌شوند یعنی الگوریتم به تدریج از دلیل اصلی توانایی آن در عبور از SA منتقل می‌شود. این رفتار کلاسیک *exploration* به *exploitation* در آن‌ها گیر می‌کند Hill Climbing هایی است که local optimum

پارامترهای حساس:

- نزدیک‌تر به 1 یعنی کوولینگ کندتر و جستجوی دقیق‌تر، اما زمان اجرای بیشتر. 0.995 تعادل alpha خوبی است.
- ها (که در مقیاس ده‌ها واحد هستند به دلیل delta باید آن قدر بزرگ باشد که در ابتدا اکثر T_start وزن 100 برای هدف بدون پوشش) قابل قبول باشند.

7. الگوریتم‌های امتیازی

Genetic Algorithm 7.1

هر فرد یک لیست از مختصات سنسورهاست. اپراتورها: **tournament selection** با اندازه‌ی 3، **uniform crossover** روی اجتماع مجموعه‌های سنسور دو والد با اندازه‌ی فرزند به صورت تصادفی بین دو والد، **mutation** با احتمال 0.3 که خود همان `get_neighbor` کلاس پایه را فراخوانی می‌کند (سازگاری اپراتورها)، و **elitism** با 2 نخبه در هر نسل.

Beam Search (Local Beam) 7.2

حالت موازی نگه می‌داریم. در هر دور، هر حالت تعدادی همسایه `beam_width`، به جای یک نقطه‌ی فعلی

حالت کم‌هزینه‌تر برای دور بعد انتخاب `beam_width` می‌سازد، همه با هم در یک استخر جمع می‌شوند، و که تضمین کند روند هرگز بدتر (elitism نوعی) فعلی هم در رقابت می‌ماند `beam` می‌شوند. خود نمی‌شود.

Tabu Search 7.3

در هر گام، بهترین همسایه‌ی غیر *تابو* انتخاب می‌شود حتی اگر بدتر از حالت فعلی باشد و سپس حالت فعلی اضافه می‌شود تا برای مدتی نتوانیم به آن (`tabu_tenure=20` با طول حداکثر `deque`) به فهرست *تابو* اجازه‌ی انجام `aspiration` برگردیم. اگر یک حرکت *تابو* منجر به بهبود بهترین جواب کل اجرا شود، با معیار `local optima` در فرار از Hill Climbing پیدا می‌کند. این مکانیزم در عمل بسیار مؤثرتر از سایدوی مووز (به نتایج بخش 8 نگاه کنید) است.

8. نتایج تجربی

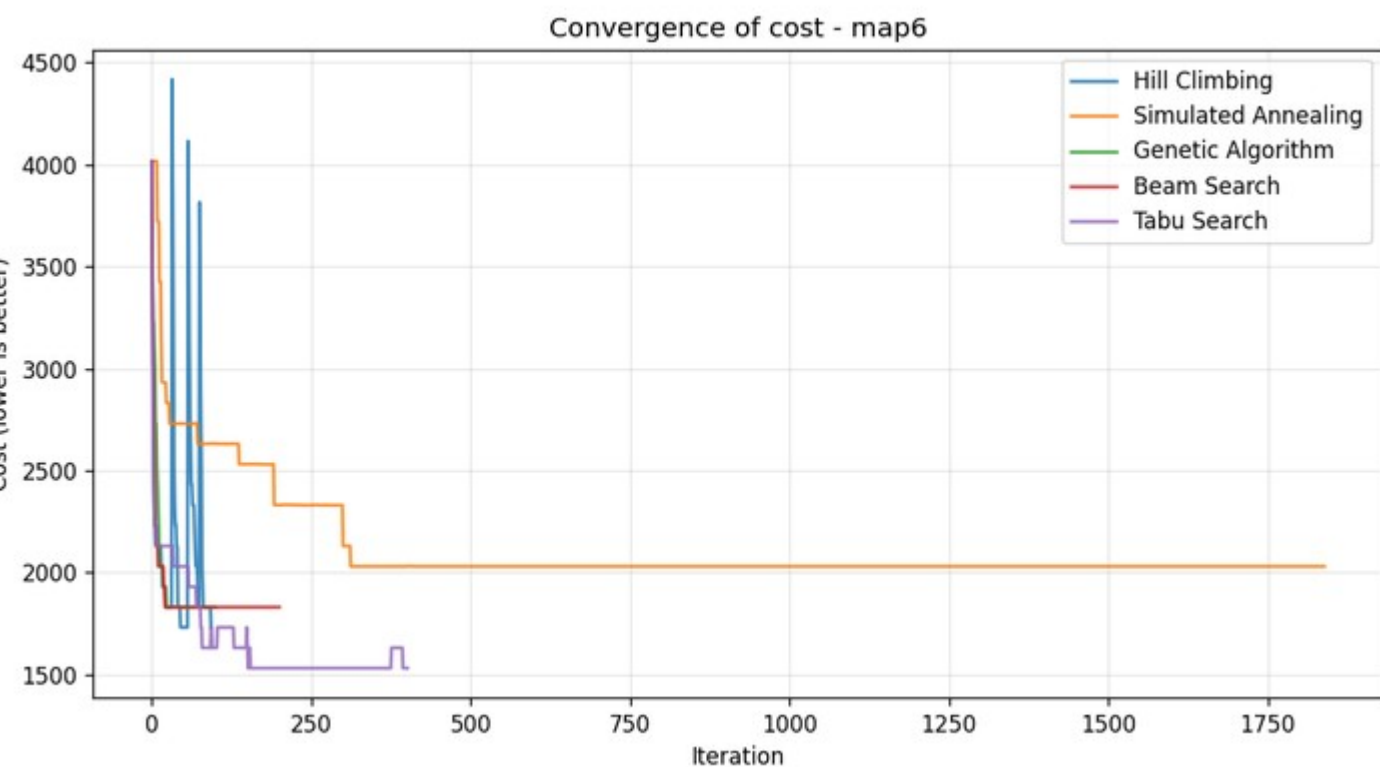
هر الگوریتم در 3 `seed` مستقل روی هر نقشه اجرا شد. در همه‌ی اجراها، حالت اولیه‌ی یکسانی استفاده می‌شود تا مقایسه‌ی منصفانه برقرار باشد. معیارها: میانگین هزینه‌ی نهایی (کمتر بهتر است)، بهترین هزینه‌ی دیده‌شده، درصد پوشش (نسبت اهداف تحت نظر به کل اهداف)، و میانگین تعداد سنسورهای استفاده‌شده.

نقشه	الگوریتم	میانگین هزینه	بهترین هزینه	پوشش (%)	تعداد سنسور
map1 (31 هدف، شعاع 2، سقف 20)	Hill Climbing	73,3	70,0	100,0	14,7
	Simulated Annealing	71,7	70,0	100,0	14,3
	Genetic Algorithm	141,7	71,0	97,8	14,7
	Beam Search	78,3	75,0	100,0	15,7
	Tabu Search	73,3	65,0	100,0	14,7

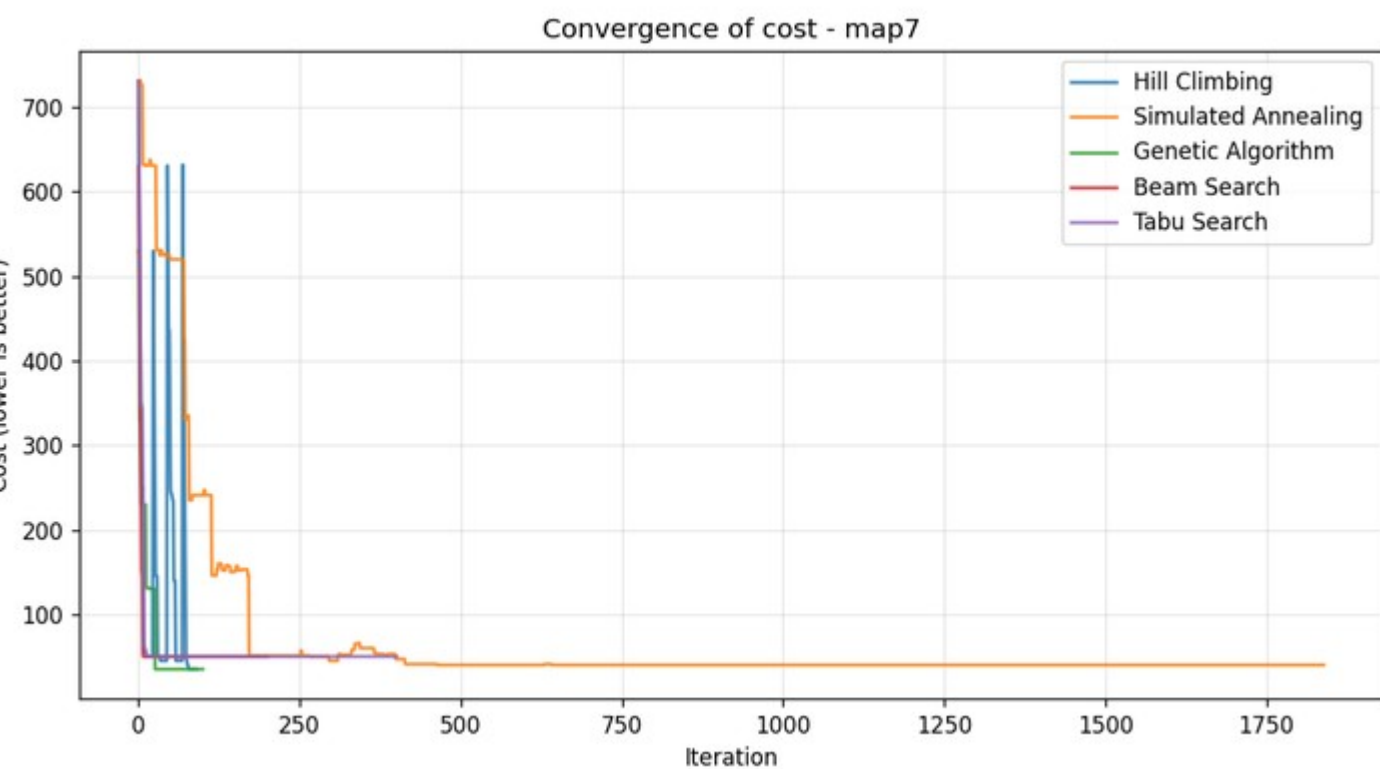
نقشه	الگوریتم	میانگین هزینه	بهترین هزینه	پوشش (%)	تعداد سنسور
map3 (40 هدف، شعاع 3، سقف 5)	Hill Climbing	260,0	227,0	94,2	5,0
	Simulated Annealing	459,7	326,0	89,2	5,0
	Genetic Algorithm	227,0	227,0	95,0	5,0
	Beam Search	227,0	227,0	95,0	5,0
	Tabu Search	227,0	227,0	95,0	5,0
map4 (11 هدف، شعاع 2، سقف 4)	Hill Climbing	220,0	220,0	81,8	4,0
	Simulated Annealing	286,7	220,0	75,8	4,0
	Genetic Algorithm	220,0	220,0	81,8	4,0
	Beam Search	220,0	220,0	81,8	4,0
	Tabu Search	253,3	220,0	78,8	4,0
map6 (53 هدف، شعاع 3)	Hill Climbing	1830,0	1730,0	66,0	6,0
	Simulated	1997,0	1830,0	62,9	6,0

نقشه	الگوریتم	میانگین هزینه	بهترین هزینه	پوشش (%)	تعداد سنسور
4، سقف 6)	Annealing				
	Genetic Algorithm	1797,3	1630,0	66,7	6,0
	Beam Search	1797,0	1730,0	66,7	6,0
	Tabu Search	1597,0	1531,0	70,4	6,0
map7 (12 هدف، شعاع 5، سقف 12)	Hill Climbing	45,0	40,0	100,0	9,0
	Simulated Annealing	36,7	35,0	100,0	7,3
	Genetic Algorithm	40,0	40,0	100,0	8,0
	Beam Search	45,0	45,0	100,0	9,0
	Tabu Search	51,7	45,0	100,0	10,3

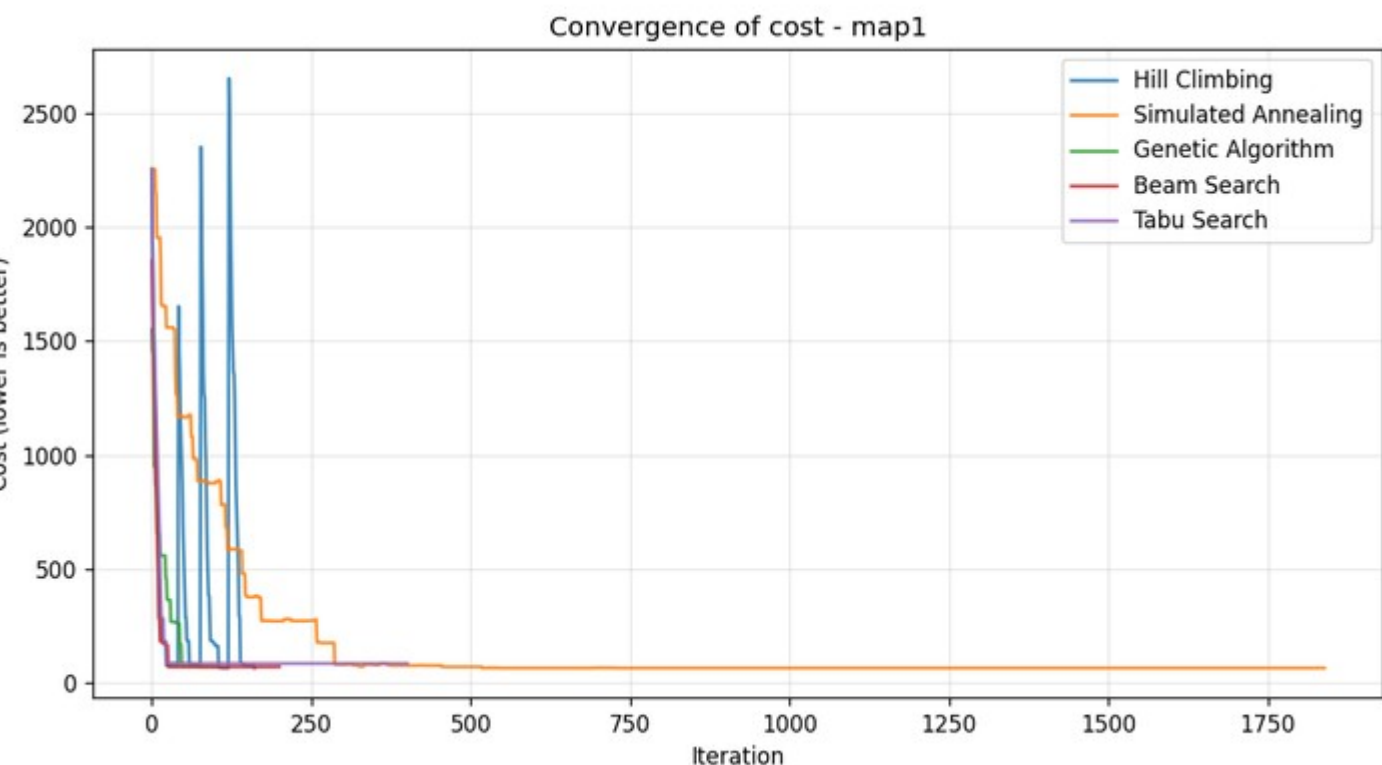
جدول 1. خلاصه‌ی نتایج تجربی روی 5 نقشه. ردیف‌های هایلایت‌شده، بهترین الگوریتم آن نقشه را نشان می‌دهند.



به‌وضوح به ناحیه‌ای پایین‌تر از سایر Tabu Search. شکل map6. 1. منحنی همگرایی هزینه روی الگوریتم‌ها می‌رسد.



با کاهش تدریجی دما، در پایان Simulated Annealing map7. شکل 2. منحنی همگرایی هزینه روی به چیدمانی با سنسور کمتر می‌رسد.



اکثر الگوریتم‌ها به سرعت به ناحیه‌ی پوشش کامل map1. شکل 3. منحنی همگرایی هزینه روی می‌رسند.

9. تحلیل

9.1 تفاوت الگوریتم‌ها

- **Hill Climbing** سریع‌ترین الگوریتم برای رسیدن به یک جواب نسبتاً خوب است، اما در نقشه‌های به سادگی گیر می‌کند (مثل map6) دشوار.
- **Simulated Annealing** با local optimum در نقشه‌هایی که فضای جواب ساختار «بسیاری» دارند بسیار خوب کار می‌کند نتیجه‌ی نمونه‌ی روشن این رفتار (به جای 35 45) map7 اختلاف کم» دارند بسیار خوب کار می‌کند نتیجه‌ی است.
- **Genetic Algorithm** (مثل map3) روی نقشه‌هایی با بودجه‌ی سنسور کم و اهداف متمرکز crossover عملکرد عالی دارد، چون خوبی بین زیرجل‌ها انجام می‌دهد.

- موازی است که با هم رقابت می‌کنند پایدار است اما HC در عمل شبیه چند **Beam Search** فرار نمی‌کند local optimum از Tabu یا SA به اندازه‌ی برترین جواب را داد. دلیل آن (مثل map6) روی نقشه‌های دشوار با اختلاف زیاد **Tabu Search** این است که حافظه‌ی کوتاه‌مدت اجازه می‌دهد بدون ترس از چرخه، گاهی به حالت‌های بدتر مهاجرت کنیم.

9.2 تأثیر پارامترها

- می‌شود؛ HC به 0,99 یا کمتر باعث کوولینگ خیلی سریع و رفتار شبیه alpha کاهش SA: در ها چند برابر می‌شود iteration افزایش به 0,999 کیفیت را بهتر می‌کند اما تعداد
- از 25 به 50 کیفیت را بهبود می‌بخشد، اما با هزینه‌ی محاسباتی k_neighbors افزایش HC: در بیشتر؛ حد بهینه به نقشه وابسته است.
- بسیار حساس است؛ مقدار خیلی کوچک، چرخه را اجازه می‌دهد و tabu_tenure تنظیم Tabu: در مقدار خیلی بزرگ، تنوع جستجو را خفه می‌کند. 20 روی نقشه‌های ما تعادل خوبی داشت
- به‌تنهایی برای تنوع crossover، در عمل لازم بود با مقدار کمتر mutation_rate=0.3 GA: در کافی نبود.

9.3 مشکل Local Optimum

خیلی شدید (فقط 4 سنسور برای 11 هدف: map4 مثل) این مسئله در نقشه‌های با سقف سنسور تنگ است. الگوریتم می‌تواند به‌سادگی در چیدمانی گیر کند که چند هدف را پوشش می‌دهد ولی به‌جای پوشش گروه دیگری از اهداف، روی همان گروه فعلی متمرکز می‌ماند. راهکارهای ما برای این مشکل

• **HC:** سایدوی مووز + restart های تصادفی.

• **SA:** پذیرش احتمالی حالت‌های بدتر.

• **Tabu:** فهرست حافظه‌ی کوتاه‌مدت.

• **GA: mutation** + جمعیت متنوع.

9.4 رفتار روی نقشه‌های مختلف

• **نقشه‌های بزرگ و باز (map5):** همه‌ی الگوریتم‌ها تقریباً یکسان عمل می‌کنند چون فضای جواب کم چالش است.

• **نقشه‌های با موانع فراوان (map4):** کشف local optimum سخت است و الگوریتم‌های random-restart برتری دارند.

• **نقشه‌های با تراکم اهداف بالا (map3، map6):** پراکنده کردن سنسورها سخت است؛ Tabu و GA بهترین عملکرد را نشان می‌دهند.

• **نقشه‌های با شعاع پوشش بزرگ (map7، شعاع 5):** SA به‌خوبی می‌تواند تعداد سنسورها را با حفظ 100٪ پوشش کم کند.

9.5 کیفیت Neighbor Function و نقش add/remove

طراحی همسایگی، در عمل مهم‌ترین ابزار ما برای مدیریت trade-off کیفیت/مصرف است. آزمایش‌های اضافی نشان داد:

• اگر فقط move را داشته باشیم: الگوریتم به اندازه‌ی state اولیه گیر می‌کند و هرگز نمی‌تواند تعداد سنسور را کم/زیاد کند یعنی trade-off اصلاً مدیریت نمی‌شود.

- هرگز به 7 یا 8 سنسور نمی‌رسند و همگی روی 9 یا map7 را حذف کنیم: نتایج روی remove اگر (که به 7,3 رسید SA مقایسه با) بیشتر متوقف می‌شوند.
- با تعداد کمی سنسور شروع کند، الگوریتم هرگز initialize_state را حذف کنیم: اگر add اگر پوشش کامل را به دست نمی‌آورد.

پس هر سه عملیات لازم و هم‌افزا هستند.

10. نحوه‌ی اجرا

```
cd project
pip install -r requirements.txt # شامل pygame, numpy, matplotlib
python main.py # اجرای هر 5 الگوریتم روی map1
```

برای تست روی نقشه‌های دیگر، در فایل main.py آرگومان GridWorld("map1") را تغییر دهید.

11. جمع‌بندی

در این پروژه پنج الگوریتم جستجوی محلی روی مسئله‌ی جانمایی سنسورها پیاده‌سازی و مقایسه شدند. نتایج تجربی نشان می‌دهد که:

1. هیچ الگوریتمی روی تمام نقشه‌ها برنده نیست (موافق با قضیه‌ی No Free Lunch).

2. طراحی دقیق تابع همسایگی خصوصاً پشتیبانی از add/remove به‌اندازه‌ی خود الگوریتم در کیفیت جواب نهایی مؤثر است.

3. وزن‌گذاری در تابع هزینه نقش حیاتی در تشخیص اولویت اهداف (پوشش در برابر تعداد سنسور) دارد.

توصیه Tabu و SA مثل local optimum برای تولید جوابی پایدار، استفاده از روش‌های آگاه به 4. می‌شود.